# Top 30 ReactJS Interview Questions

@Aman_Views

**1. Why does useEffect sometimes run more than once, and how do you prevent unnecessary or infinite re-runs?**

Example

```
useEffect(() => {
    // side effect
    fetchData();
    return () => cancelRequest();
}, [userId]); // runs when userId changes
```

- In React 18 Strict Mode (development), useEffect runs twice on mount to help detect side-effect bugs.
- It also runs again whenever a value in the dependency array changes.
- To prevent unnecessary or infinite re-runs:
  - Provide the correct dependency array.
  - Memoize functions/values with useCallback / useMemo.
  - Avoid creating new objects/arrays in render.
  - Use empty array [] to run only once (on mount).

---

**2. What happens if you forget a dependency in the useEffect dependency array?**

- The effect will not run when that dependency changes.
- This can lead to stale data, incorrect behavior, or bugs that are hard to track.
- ESLint (react-hooks/exhaustive-deps) helps catch missing dependencies.

---

**3. What is a stale closure in React, and how can it cause bugs?**

Example (stale closure)

```
useEffect(() => {
   const id = setInterval(() => {
      console.log(count); // may log old count
   }, 1000);
   return () => clearInterval(id);
}, []); // count is missing -> stale value
```

- A stale closure happens when a function "remembers" old values from when it was created.
- It can cause bugs because the function uses outdated state or props.
- Fix by adding dependencies, using functional updates, or useRef to hold latest values.

---

**4. Why does React sometimes batch multiple state updates into a single re-render?**

- Batching improves performance by reducing the number of re-renders.
- In React 18, automatic batching is enabled for updates inside events, promises, timeouts, and more.
- React waits until the end of the event loop, then updates the state once.

---

**5. Why doesn't calling setState immediately update the state variable?**

Example

```
setCount(count + 1);
console.log(count); // still old value
useEffect(() => {
   console.log(count); // updated value
}, [count]);
```

- setState is asynchronous.
- React schedules the update and may batch it with other updates.
- The state variable you read right after setState is still the old value.
- Use useEffect or the updater function to work with the latest value.

# Top 30 ReactJS Interview Questions

@Aman_Views

## 6. What causes a React component to re-render?

Causes :

1. State update (useState, useReducer)
2. Props change from parent
3. Context value change
4. Parent re-render (even if props are same, unless memoized)
5. forceUpdate() (class components)

- When any of the above happens, React schedules a re-render for that component.
- If props/state/context values are shallowly different, React re-renders the component.
- React uses reconciliation to figure out what needs to be updated in the DOM.

## 7. Why can useMemo and useCallback actually make your application slower?

Reasons :

- They add extra memory usage.
- They add comparison cost (deps check).
- Overusing them adds unnecessary complexity.
- Only useful when it actually prevents expensive re-computations or re-renders.

- If the computation is cheap, memoization overhead can be higher.
- If dependencies change frequently, memoization won't help.
- Premature optimization can make code harder to read and maintain.

## 8. What is the difference between useMemo, useCallback, and React.memo()?

| Hook / API | What it does | Returns | Use for |
|---|---|---|---|
| useMemo | Memoizes a computed value | Memoized value | Expensive calculations (values) |
| useCallback | Memoizes a function reference | Memoized function | Passing callbacks to child components |
| React.memo() | Memoizes a component | Memoized component | Prevent re-render of functional components |

- useMemo and useCallback are Hooks used inside components.
- React.memo() is a HOC (Higher Order Component).
- All help in performance optimization but used in different scenarios.

## 9. Why are key props important in React lists, and why is using an array index as a key risky?

Why key is important?

- Helps React identify which items changed, added, or removed.
- Improves performance.
- Maintains component state correctly.

Why index as key is risky?

- If list order changes, state may get mixed up.
- Can cause unnecessary re-renders.
- Leads to bugs in dynamic lists (insert, delete, sort).

- Always use a stable and unique id as key whenever possible.
- Avoid using index unless the list is static (no reordering, no insertion/deletion).

## 10. What is the difference between useEffect and useLayoutEffect?

| Hook | When it runs | Use case | Does it block paint? |
|---|---|---|---|
| useEffect | After paint (async) | Data fetching, subscriptions, logging, non-DOM work | No |
| useLayoutEffect | Before paint (sync) | DOM measurements, layout manipulations | Yes |

- useLayoutEffect runs synchronously after DOM mutations but before the browser paints.
- Use useLayoutEffect only when you need to read layout or make layout changes.
- Overuse of useLayoutEffect can hurt performance.

# Top 30 ReactJS Interview Questions

@Aman_Views

## 11. What are the Rules of Hooks, and why can't Hooks be called inside loops or conditions?

Rules of Hooks :

1. Only call Hooks at the top level.
2. Only call Hooks from React functions (function components or custom Hooks).
3. Don't call Hooks inside loops, conditions, or nested functions.

- Hooks rely on call order to associate state with the correct component.
- If Hooks are called conditionally or inside loops, the order changes between renders.
- This breaks React's internal state mapping and leads to bugs.
- Always keep Hooks at the top level of your component or custom Hook.

## 12. What is the difference between useState and useReducer, and when would you choose useReducer?

| Feature | useState | useReducer |
|---|---|---|
| Purpose | Manage simple state | Manage complex state with multiple sub-values |
| State update | Direct value | Dispatch actions |
| Logic location | Scattered in component | Centralized in reducer function |
| Best for | Simple state | Complex state / many related updates |

- Use useState for simple state (string, number, boolean, object).
- Use useReducer when state logic is complex, involves multiple sub-values, or next state depends on action type.
- useReducer makes state transitions predictable and easier to test.

## 13. Why should you use a functional state update like setCount(c => c + 1)?

Example

```
setCount(c => c + 1);  // ☑ correct
setCount(count + 1);   // ✕ may use stale value
```

- Functional updates always use the latest state value.
- Avoids stale closure issues in async updates or effects.
- Important when multiple state updates are batched together.

## 14. What is useRef, and how is it different from useState?

| Feature | useState | useRef |
|---|---|---|
| Stores | State value | Mutable value (ref) |
| Triggers re-render | Yes | No |
| Update method | setState() | ref.current = value |
| Use case | UI/state changes | DOM access, timers, mutable values |

- useRef returns a mutable object with a .current property.
- You can change .current without causing re-render.
- Commonly used for DOM references, timers, storing previous values, etc.

## 15. Why does updating a useRef value not cause a re-render?

Example

```
const countRef = useRef(0);
countRef.current = countRef.current + 1;
// No re-render happens
```

- React only re-renders when state or props change.
- useRef updates do not trigger a re-render.
- The value is stored outside of the rendering cycle.
- This makes useRef perfect for storing values that don't affect UI directly.

# Top 30 ReactJS Interview Questions

@Aman_Views

## 16. What is a custom Hook, and how do you decide when to create one?

Example

```
function useWindowSize() {
  const [size, setSize] = useState({
    width: window.innerWidth,
    height: window.innerHeight
  });
  // logic ...
  return size;
}
```

- A custom Hook is a JavaScript function whose name starts with "use" and can call other Hooks.
- It lets you extract and reuse stateful logic across components.
- Create a custom Hook when you see repeated logic that uses state, effects, or other Hooks.
- Helps in code reuse, separation of concerns, and easier testing.
- Keep Hooks small, focused, and composable.

## 17. How does useContext work, and what performance problem can it cause?

Example

```
const ThemeContext = createContext();
// Provider
<ThemeContext.Provider value={theme}>
  {children}
</ThemeContext.Provider>
// Consumer
const theme = useContext(ThemeContext);
```

- useContext lets components read values from a Context without prop drilling.
- When the context value changes, all consuming components re-render.
- Performance problem: Even components that don't use the changed part will re-render.
- Fixes: Split context, memoize value, useContextSelector (or libraries like Zustand, Redux).

## 18. How would you share state between sibling components without prop drilling?

Approaches

1. Lift state up to the nearest common parent.
2. Use Context API.
3. Use a global state manager (Redux, Zustand, Recoil, Jotai, etc.)
4. Use an event bus / pub-sub pattern.

- Prop drilling can make code hard to maintain.
- Context is good for medium-level sharing.
- For complex or large-scale apps, prefer a global state solution like Redux or Zustand.
- Choose based on app size, update frequency, and team preference.

## 19. What is reconciliation in React, and how does React decide what needs to change in the DOM?

Key Points

- Reconciliation is the process React uses to update the DOM efficiently.
- React compares the previous and next UI (Virtual DOM).
- Uses a diffing algorithm with these heuristics:
  - Elements of different types produce different trees.
  - Keys help identify stable elements in lists.

- React builds a new Virtual DOM on every render.
- It compares the new tree with the previous one.
- Only the minimal set of changes are applied to the real DOM.
- Keys help React match items between renders for better performance and correct state preservation.

## 20. What is the Virtual DOM, and why doesn't "Virtual DOM = faster" tell the whole story?

Key Points

- Virtual DOM is a lightweight JS representation of the real DOM.
- It allows React to batch updates and minimize direct DOM manipulations.
- But "Virtual DOM = faster" is an oversimplification.

- Performance depends on many factors: unnecessary re-renders, heavy computations, large components, poor state management, etc.
- If you re-render too often or create new objects functions every render, it will still be slow.
- Write efficient components, use memoization, and profile your app.

# Top 30 ReactJS Interview Questions

@Aman_Views

## 21. What happens when a parent component re-renders? Do all of its children re-render?

Example

A parent re-render does not force all children to re-render.

- Children re-render only if:
  - Their props change.
  - They use state or context that changed.

- By default, all child components re-render when their parent re-renders.
- React uses reconciliation to decide whether a child actually needs to update in the DOM.
- You can prevent unnecessary child re-renders using React.memo(), useMemo, or useCallback.
- So, parent re-render doesn't automatically mean all children will re-render.

## 22. How does React preserve state between renders, and when does React reset a component's state?

Example

- State is preserved between renders as long as the component keeps the same position in the UI tree and the same key.
- React resets a component's state when:
  - The component unmounts.
  - The component's key changes.

- React keeps the state of a component between renders by matching the component's type and position in the tree.
- If a component is removed from the tree (unmount), its state is lost.
- Changing the component's key also makes React treat it as a new component, so the state is reset.

## 23. What is the difference between rendering and committing in React?

Example

- Rendering: means React is preparing the UI (creating the virtual DOM).
- Committing: means React is applying the changes to the real DOM.

- Rendering is a "pure" phase — it calculates what the UI should look like.
- Committing is the phase where React applies those changes to the DOM and runs side effects.
- Rendering can happen multiple times, but committing happens only once per update.

## 24. Why can changing a component's key cause its state to reset?

Example

- The key prop helps React identify a component's identity.
- When the key changes, React treats it as a new component.
- As a result, the previous state is discarded and the component is re-created with fresh state.

- React uses the key to keep track of components between renders.
- Changing the key tells React: "This is a different component now."
- Therefore, React unmounts the old component and mounts a new one, resetting its state.

## 25. What is the difference between controlled and uncontrolled components?

Example

- Controlled component: its value is controlled by React state (through props).
- Uncontrolled component: manages its own state internally (using refs).

- Controlled components: React controls the value through state and onChange handlers.
- Uncontrolled components: maintain their own internal state and don't rely on React state.
- Controlled components give you more control, while uncontrolled components are simpler.

# Top 30 ReactJS Interview Questions

@Aman_Views

## 26. How would you optimize a React application that has unnecessary re-renders?

**Answer / Explanation**

1. Use `React.memo()` for memoizing components.
2. Use `useMemo()` and `useCallback()` for expensive calculations and functions.
3. Avoid inline functions and objects in JSX.
4. Lift state up only when necessary.
5. Use `useRef()` for values that don't need to trigger re-render.
6. Split large components into smaller ones.
7. Use virtualization for long lists (e.g., react-window).

- Re-renders happen when props, state, or context change.
- The goal is to re-render only what actually needs to update.
- Profile your app using React DevTools Profiler to find bottlenecks.
- Optimize step by step — measure, optimize, then measure again.

## 27. What is code splitting in React, and how does React.lazy() work with Suspense?

**Answer / Explanation**

- Code splitting is the technique of splitting your code into smaller chunks which are loaded on demand.
- `React.lazy()` lets you import a component lazily (asynchronously).
- `Suspense` is used to show a fallback UI while the lazy component is loading.

**Example**

```
const MyComp = React.lazy(() => import("./MyComp"));
function App() {
  return (
    <Suspense fallback={<div>Loading...</div>}>
      <MyComp />
    </Suspense>
  );
}
```

- This reduces the initial bundle size.
- The component is loaded only when it's needed.
- Improves performance by reducing load time and memory usage.

## 28. What is the difference between Context API, Redux, and other external state-management solutions?

**Comparison**

| Solution | When to use | Pros | Cons |
|---|---|---|---|
| Context API | Small to medium apps, simple state | Built-in, easy to use, no extra libs | Can cause re-renders, not great for large or complex state |
| Redux (Toolkit) | Large apps, complex state, many updates | Predictable, powerful devtools, middlewares | Boilerplate (less with RTK), learning curve |
| Other (Zustand, Jotai, Recoil, etc.) | Modern apps, fine-grained state, performance focus | Less boilerplate, better performance | Smaller ecosystem (than Redux) |

- Context is good for theme, user info, or infrequently changing data.
- Redux is ideal for complex, global state with many interactions.
- Other libraries focus on simplicity, minimal re-renders, and better performance.
- Choose based on app size, team familiarity, and requirements.

## 29. What are React Server Components, and how are Server Components different from Client Components?

**Key Points**

- Server Components (RSC) run on the server, not in the browser.
- They can fetch data directly and don't send JS to the client.
- Client Components run in the browser and handle interactivity (useState, useEffect, etc.).
- RSCs reduce bundle size and improve performance.

| Feature | Server Components | Client Components |
|---|---|---|
| Where they run | On the server | In the browser |
| Can use Hooks | No (only async/await for data) | Yes (useState, useEffect, etc.) |
| Access to data | Directly on the server | Needs fetch / API call |
| Bundle size | Not sent to client | Sent to client |
| Interactivity | No interactivity | Fully interactive |

## 30. What is hydration, and what can cause a hydration mismatch in a React application?

**Answer / Explanation**

- Hydration is the process where React attaches event listeners to the server-rendered HTML and makes it interactive.
- A hydration mismatch happens when the HTML generated on the server is different from what the client renders on the first render.

Common causes of hydration mismatch:

1. Using non-deterministic values like `Date.now()`, `Math.random()`.
2. Different data between server and client (e.g., user agent, localStorage, cookies).
3. Conditional rendering that depends on `window` or browser APIs.
4. Invalid HTML structure.
5. Using `useEffect` for initial UI differenctes instead of server data.

Fix: Ensure the first render is the same on both server and client.